

A Learned "Physics Sense" for Robot Planning

Latent dynamics model + planning for block-pushing on the LangTable simulator — project summary

The Problem This Solves

A robot arm pushes blocks around a tabletop with a peg-shaped tip toward a goal described in plain English, e.g. *"push the green star close to the green cube."* To pick a good sequence of pushes, a planner needs a way to score candidate action sequences: *how close does this sequence get me to the goal?*

The project's earlier approach (still in the repository, described below) scored each candidate by asking a large vision-language model a yes/no question — "is the green star close to the green cube?" — once per candidate, per planning step. That works, but it is slow: a full forward pass through a multi-billion-parameter model for every single candidate trajectory the planner considers.

This project's goal is to replace that expensive per-candidate scoring with something far cheaper: a small model that predicts, directly in a compact numerical representation, what the scene will look like after a proposed sequence of pushes — without rendering or simulating anything. Comparing that prediction to a similarly-encoded goal description is nearly free, so a planner can evaluate hundreds of candidates per step instead of a handful.

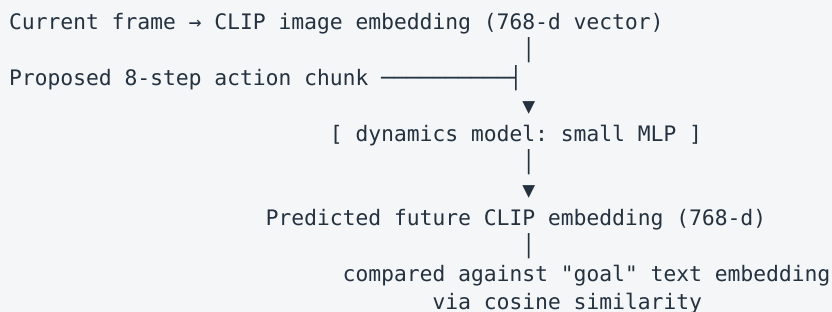
The Task: "blocktoblock" in LangTable

LangTable is a 2D tabletop simulator (PyBullet-based). A robot moves a peg-shaped end effector across the table to push around eight fixed colored blocks (e.g. `green_star`, `blue_moon`, `red_cube`). In the **blocktoblock** task, each episode designates one block as the one to move (the "start block") and another as its destination (the "target block"); success means pushing the start block until it is touching, or very close to, the target block. The project trains and evaluates against roughly 5,000 recorded demonstration episodes of this task (plus a smaller supplementary set of failure/suboptimal episodes used in later training runs), each a sequence of camera frames paired with the 2D push actions taken at every timestep.

The Core Idea: Predict Meaning, Not Pixels

Rather than predicting future camera frames (expensive, and full of irrelevant visual detail), the model predicts a future **CLIP embedding** — a fixed-length vector that CLIP (a general-purpose vision-language model from OpenAI/LAION) produces to summarize the semantic content of an image or a sentence, such that semantically similar images and sentences land near each other in the same vector space. The dynamics model's job is: given the CLIP embedding of the *current* camera frame, plus a short chunk of proposed push actions, predict the CLIP embedding of the frame *several steps in the future*, after those pushes are executed.

Because goal instructions ("push X close to Y") can also be turned into CLIP text embeddings, the planner never needs a rendered image at all during planning — it can compare the model's predicted future embedding directly against a goal-text embedding using simple vector similarity (cosine similarity), and pick whichever candidate action sequence gets closest.



How the Model Is Trained: Contrastive Learning on Scene Statements

The dynamics model isn't trained to reproduce an exact target vector (a plain regression loss would encourage a "blurry average" prediction and give no signal about which differences in the scene actually matter). Instead it's trained **contrastively**: for a given predicted embedding, the training objective simply asks the model to rank one correct description of the resulting scene above a large pool of incorrect ones.

1. Turning the scene into text

For every timestep of every demonstration, the exact positions of the blocks and the peg are known from the simulator, so a short factual sentence can be generated automatically, e.g.:

"The green star is touching the green cube. The peg is not touching the green star."

This statement is CLIP-text-encoded into the same 768-dimensional space as the image embeddings. That becomes the **positive** (correct) target for whatever frame occurs at that timestep.

2. Manufacturing guaranteed-false negatives

For the same timestep, the training code also generates a large batch of statements about the *same* blocks that are guaranteed to be false — e.g. claiming two blocks are touching when they are known not to be, or that the peg is touching a block it demonstrably isn't. With 8 blocks on the table, there are 56 ordered block pairs and 8 possible "peg touching" clauses, giving several hundred combinatorially-generated false statements per timestep (224 in the original scheme; 448 in the current, wider variant covering both fine-tuned CLIP towers). These are the **hard negatives**.

3. The contrastive (InfoNCE) loss

Given the model's predicted embedding, the training loss (a standard contrastive objective called InfoNCE) treats this as a classification problem: out of one correct statement and hundreds of manufactured false ones, the model must assign the highest similarity score to the correct one. This pushes the predicted embedding toward the true future scene description and away from every plausible-sounding false alternative, without ever requiring an expensive dense regression target.

Two Generations of the Model

Generation 1 — frozen CLIP + small MLP (current default)

CLIP's image and text encoders are used exactly as pretrained, with no changes. The only trainable part is a small conditioned MLP ("FiLM-MLP") that takes the current 768-dimensional CLIP image embedding and the 8-step action chunk, and outputs a predicted future 768-dimensional embedding. This is cheap to train (all CLIP embeddings can be pre-computed and cached once), and is the version with the most completed training runs and evaluation.

Generation 2 — CLIP fine-tuned end-to-end (in progress)

The newer line of work unfreezes CLIP's own image encoder *and* text encoder and trains them jointly with the MLP, all under the same contrastive loss described above. The image encoder, the MLP, and the text encoder are updated together by the same gradient signal, rather than treating CLIP as a fixed, generic feature extractor. The idea is that CLIP's off-the-shelf embeddings weren't designed to capture this specific task's fine-grained notion of "touching" — end-to-end fine-tuning lets the whole pipeline specialize its representation of the scene around exactly the distinctions the planner cares about.

This second variant is the more expensive of the two: because the text encoder is now trainable, its embeddings can no longer be pre-computed and cached — every one of the several hundred candidate statements has to be re-encoded on the fly, every training step, for both towers simultaneously. It has also been extended to train on a combined dataset: the original ~5,000 successful demonstration episodes plus roughly 1,675 additional "suboptimal" episodes (mostly failed attempts), so the model sees a wider range of scene configurations, not just ones that end in success.

How the Model Is Used at Planning Time

Once trained, the dynamics model is used inside a sampling-based planner (implemented as either a cross-entropy-method-style optimizer or its sibling textbook CEM variant). At each planning step, the planner:

1. Samples a population of candidate 8-step action chunks;
2. Runs each candidate through the dynamics model — in latent-embedding space, with no rendering or physics simulation involved — to get a predicted future embedding;
3. Scores each candidate by cosine similarity between its predicted embedding and the goal-text embedding;
4. Refits its sampling distribution toward the best-scoring candidates and repeats for a few iterations;
5. Executes the single best-scoring action chunk in the real simulator, then repeats the whole process from the new state.

Because every step of this loop is a handful of small matrix multiplications rather than a vision-language-model forward pass, the planner can afford to evaluate hundreds of candidates per planning step — something that would be far too slow with the original VLM-reward-model approach this project builds on.

Status

The frozen-CLIP + MLP model (Generation 1) is fully trained and has been evaluated across many checkpoints, both on held-out prediction accuracy and on closed-loop rollouts in the real simulator. The end-to-end CLIP fine-tuning line (Generation 2) is the actively evolving part of the project, including the recent extension to train on combined expert and suboptimal trajectory data. Ongoing evaluation work is focused on confirming that a model's own predicted scores translate into genuinely better pushes in the real simulator, not just numerically higher self-reported scores.

Project repository: [SWM_copy/](#) (this document) and legacy [swm/](#) (the earlier VLM-reward-model approach this project supersedes).